APACHE WEB SERVER

Ubuntu 20.04 • 22.04 • 24.04

Practical Configuration Guide

HTTP only — without SSL / HTTPS

Topics covered

- Install and manage Apache on Ubuntu
- Host a single website using a Virtual Host
- Host multiple websites on one server
- Configure Apache as a reverse proxy and load balancer
- Enable and disable sites, verify configuration, and inspect logs
- RHEL/CentOS versus Ubuntu command reference

This guide uses Apache on port 80 (HTTP). SSL certificates, Certbot, HTTPS virtual hosts, and certificate renewal are intentionally excluded.

1. Install Apache Web Server on Ubuntu

Ubuntu uses the **apache2** package and service name. Run the following commands:

Update package repository and install Apache

```
sudo apt update
sudo apt install apache2 -y
```

Start, enable, and check Apache

```
sudo systemctl start apache2
sudo systemctl enable apache2
sudo systemctl status apache2
```

Check Apache version

```
apache2 -v
```

Test in a browser

Open **http://YOUR_SERVER_PUBLIC_IP**. You should see the Apache2 Ubuntu Default Page.

Network requirement: Allow inbound TCP port 80 in the cloud security group and the server firewall, if enabled.

2. Configure Apache for a Single Website

Example domain: **devlinux.xyz**. Replace this example domain with your own domain.

Step 1 — Create the website directory

```
sudo mkdir -p /var/www/devlinux.xyz
```

Step 2 — Create the website home page

```
sudo vim /var/www/devlinux.xyz/index.html
```

Example HTML content:

```
<!doctype html>
<html>
<head>
    <title>DevLinux Website</title>
</head>
<body>
    <h1>Welcome to DevLinux Website</h1>
    <p>Apache Web Server is working successfully!</p>
</body>
</html>
```

Step 3 — Create a Virtual Host configuration

```
sudo vim /etc/apache2/sites-available/devlinux.xyz.conf
```

```
<VirtualHost *:80>
    ServerName devlinux.xyz
    ServerAlias www.devlinux.xyz

    DocumentRoot /var/www/devlinux.xyz
    DirectoryIndex index.html

    ErrorLog ${APACHE_LOG_DIR}/devlinux_error.log
    CustomLog ${APACHE_LOG_DIR}/devlinux_access.log combined
</VirtualHost>
```

Step 4 — Enable the website

```
sudo a2ensite devlinux.xyz.conf
```

Step 5 — Test and apply the configuration

```
sudo apache2ctl configtest
sudo systemctl reload apache2
```

Expected configuration-test output: **Syntax OK**.

Step 6 — Open the website

Visit **http://devlinux.xyz**. The domain's DNS A record must point to the server's public IP address, and TCP port 80 must be reachable.

3. Configure Apache for Multiple Websites

Apache Virtual Hosts allow multiple domains to share one Apache server. Each website gets its own document root and Virtual Host configuration.

Ubuntu Apache directory structure

Purpose	Ubuntu path
Main configuration	/etc/apache2/apache2.conf
Available websites	/etc/apache2/sites-available/
Enabled websites	/etc/apache2/sites-enabled/
Available modules	/etc/apache2/mods-available/
Enabled modules	/etc/apache2/mods-enabled/
Website files	/var/www/
Log files	/var/log/apache2/

Website 1 — devlinux.xyz

```
sudo mkdir -p /var/www/devlinux.xyz
sudo vim /var/www/devlinux.xyz/index.html
sudo vim /etc/apache2/sites-available/devlinux.xyz.conf
```

```
<VirtualHost *:80>
    ServerName devlinux.xyz
    ServerAlias www.devlinux.xyz
    DocumentRoot /var/www/devlinux.xyz

    ErrorLog ${APACHE_LOG_DIR}/devlinux_error.log
    CustomLog ${APACHE_LOG_DIR}/devlinux_access.log combined
</VirtualHost>
```

```
sudo a2ensite devlinux.xyz.conf
```

Website 2 — website2.com

```
sudo mkdir -p /var/www/website2
sudo vim /var/www/website2/index.html
sudo vim /etc/apache2/sites-available/website2.conf
```

```
<VirtualHost *:80>
    ServerName website2.com
    ServerAlias www.website2.com
    DocumentRoot /var/www/website2

    ErrorLog ${APACHE_LOG_DIR}/website2_error.log
    CustomLog ${APACHE_LOG_DIR}/website2_access.log combined
</VirtualHost>
```

```
sudo a2ensite website2.conf
```

Website 3 — site1.com

```
sudo mkdir -p /var/www/site1/public_html
sudo vim /var/www/site1/public_html/index.html
sudo vim /etc/apache2/sites-available/site1.conf
```

```
<VirtualHost *:80>
```

```
    ServerName site1.com
    ServerAlias www.site1.com
    DocumentRoot /var/www/site1/public_html

    ErrorLog ${APACHE_LOG_DIR}/site1_error.log
    CustomLog ${APACHE_LOG_DIR}/site1_access.log combined
</VirtualHost>
```

```
sudo a2ensite site1.conf
```

Verify all Virtual Hosts

```
sudo apache2ctl configtest
sudo systemctl reload apache2
sudo apache2ctl -S
```

Access the domains using their HTTP names:

```
http://devlinux.xyz
http://website2.com
http://site1.com
```

All domain DNS records should point to the same Apache server's public IP. Create a separate index.html in each document root so that each site displays its own content.

4. Configure Apache as a Load Balancer

Apache can act as a reverse proxy and distribute incoming HTTP requests across multiple backend web servers.

Step 1 — Enable required modules

```
sudo a2enmod proxy
sudo a2enmod proxy_http
sudo a2enmod proxy_balancer
sudo a2enmod lbmethod_byrequests
```

Step 2 — Configure the load-balanced Virtual Host

```
sudo vim /etc/apache2/sites-available/website2.conf
```

```
<VirtualHost *:80>
    ServerName website2.com
    ServerAlias www.website2.com

    <Proxy "balancer://mycluster">
        BalancerMember http://13.234.21.220:80
        BalancerMember http://13.235.67.39:80
        ProxySet lbmethod=byrequests
    </Proxy>

    ProxyPass "/" "balancer://mycluster/"
    ProxyPassReverse "/" "balancer://mycluster/"

    ErrorLog ${APACHE_LOG_DIR}/website2_error.log
    CustomLog ${APACHE_LOG_DIR}/website2_access.log combined
</VirtualHost>
```

Replace the example backend IP addresses with your actual backend server addresses.

Step 3 — Test and restart Apache

```
sudo apache2ctl configtest
sudo systemctl restart apache2
```

How load balancing works

• The client sends an HTTP request to website2.com.

• Apache receives the request as the load balancer.

• Apache forwards the request to one of the configured backend servers.

• The byrequests method distributes requests according to the number of requests assigned to each worker.

Important: Both backend servers must be reachable from the load balancer on TCP port 80. Their firewall/security-group rules must allow that traffic, and both backends should serve the expected application content. This basic example does not configure health checks or session stickiness.

5. Enable and Disable Websites

Enable a website

```
sudo a2ensite devlinux.xyz.conf
```

Disable a website

```
sudo a2dissite devlinux.xyz.conf
```

List enabled and available websites

```
ls /etc/apache2/sites-enabled/
ls /etc/apache2/sites-available/
```

After changing site configuration, validate it and reload Apache:

```
sudo apache2ctl configtest
sudo systemctl reload apache2
```

6. Restart and Verify Apache

Task	Command
Start	sudo systemctl start apache2
Stop	sudo systemctl stop apache2
Restart	sudo systemctl restart apache2
Reload	sudo systemctl reload apache2
Check status	sudo systemctl status apache2
Enable at boot	sudo systemctl enable apache2
Validate configuration	sudo apache2ctl configtest
Show Virtual Host mapping	sudo apache2ctl -S

View Apache logs

```
sudo tail -f /var/log/apache2/error.log
sudo tail -f /var/log/apache2/access.log
```

Use the website-specific log files configured in each Virtual Host when troubleshooting a particular domain.

7. RHEL/CentOS vs Ubuntu — Important Commands

Task	RHEL/CentOS	Ubuntu
Install Apache	yum install httpd -y	apt install apache2 -y
Start	systemctl start httpd	systemctl start apache2
Stop	systemctl stop httpd	systemctl stop apache2
Restart	systemctl restart httpd	systemctl restart apache2
Check status	systemctl status httpd	systemctl status apache2
Enable at boot	systemctl enable httpd	systemctl enable apache2
Main config	/etc/httpd/conf/httpd.conf	/etc/apache2/apache2.conf
Virtual Hosts	/etc/httpd/conf.d/	/etc/apache2/sites-available/
Document root	/var/www/html/	/var/www/html/
Logs	/var/log/httpd/	/var/log/apache2/
Enable website	Manual configuration	a2ensite
Disable website	Manual configuration	a2dissite
Enable module	Configuration-dependent	a2enmod
Disable module	Configuration-dependent	a2dismod

8. Final Ubuntu Apache Directory Structure

```
/etc/apache2/
■■■ apache2.conf
■■■ ports.conf
■■■ sites-available/
■   ■■■ devlinux.xyz.conf
■   ■■■ website2.conf
■   ■■■ site1.conf
■■■ sites-enabled/
■   ■■■ devlinux.xyz.conf
■   ■■■ website2.conf
■   ■■■ site1.conf
■■■ mods-available/
■■■ mods-enabled/

/var/www/
■■■ devlinux.xyz/
■   ■■■ index.html
■■■ website2/
■   ■■■ index.html
■■■ site1/
    ■■■ public_html/
        ■■■ index.html

/var/log/apache2/
■■■ devlinux_error.log
■■■ devlinux_access.log
■■■ website2_error.log
■■■ website2_access.log
■■■ site1_error.log
■■■ site1_access.log
```

9. Important Points to Remember

• Ubuntu uses apache2; RHEL/CentOS uses httpd.

• Ubuntu Apache configuration is stored under /etc/apache2/.

• sites-available contains Virtual Host configuration files available to Apache.

• sites-enabled contains the enabled site configurations, usually as symbolic links.

• a2ensite enables a site; a2dissite disables a site.

• a2enmod enables a module; a2dismod disables a module.

• Virtual Hosts allow multiple websites to run on one Apache server.

• Apache can also operate as a reverse proxy and HTTP load balancer.

Scope and production note: This guide intentionally configures HTTP only on port 80. HTTPS/SSL is not configured. For public production websites, use HTTPS to protect traffic. Also configure DNS, firewall/security-group rules, file permissions, and backend availability appropriately.